UF2405. UD1. Actividad 1_Ejercicio teórico-práctico "COMPARATIVA .NET Y J2EE"

Pregunta 1. Comparativa entre .NET y J2EE como tecnologías para realizar un proyecto web.

Tanto **.NET** (desarrollado por Microsoft) como **J2EE** (Java 2 Platform, Enterprise Edition, actualmente Jakarta EE) son los dos estándares de facto en la industria para el desarrollo de aplicaciones web empresariales robustas y escalables. A continuación, se detallan sus características y diferencias.

Características de J2EE (Java EE)

- **Independencia de plataforma:** Basado en el lema *"Write Once, Run Anywhere"*. Gracias a la Máquina Virtual de Java (JVM), el mismo código compilado (.class o .war) puede ejecutarse en Windows, Linux o macOS sin modificaciones.
- **Lenguaje:** Utiliza Java, un lenguaje fuertemente tipado, orientado a objetos y con un ecosistema open-source masivo.
- **Servidores de Aplicaciones:** Existe una gran variedad de contenedores, tanto libres (Apache Tomcat, WildFly, GlassFish) como comerciales (WebLogic, WebSphere).
- **Acceso a datos:** Utiliza **JDBC** (Java Database Connectivity) como estándar para conectar con cualquier SGBD (MySQL, Oracle, H2, etc.).
- **Gestión de dependencias:** Se apoya en herramientas como **Maven** o Gradle para automatizar la compilación y descarga de librerías.

Características de .NET (Microsoft)

- **Plataforma:** Originalmente ligado a Windows, aunque las versiones modernas (.NET Core / .NET 5+) son multiplataforma y de código abierto.
- **Lenguaje:** Utiliza principalmente C# o VB.NET, compilados a un lenguaje intermedio (CIL) que se ejecuta sobre el CLR (Common Language Runtime).
- **Servidores de Aplicaciones:** Históricamente ligado a IIS (Internet Information Services), aunque ahora utiliza Kestrel en entornos multiplataforma.
- **Acceso a datos:** Utiliza **ADO.NET** y Entity Framework como ORM principal.
- **Ecosistema:** Muy integrado con el entorno de desarrollo Visual Studio y la nube de Microsoft Azure.

Tabla Comparativa Resumen

Característica	J2EE (Java EE / Jakarta EE)	.NET (Microsoft)
Lenguaje principal	Java	C#, VB.NET
Ejecución	JVM (Máquina Virtual Java)	CLR (Common Language Runtime)
Servidor Web/App	Apache Tomcat, WildFly, JBoss	IIS, Kestrel
Acceso a Datos	JDBC, JPA / Hibernate	ADO.NET, Entity Framework
Coste / Licencia	Open Source / Estándar abierto	Open Source (.NET moderno) / Propietario (histórico)

Aplicación en nuestro Mini-Proyecto:

Para este proyecto hemos elegido **J2EE** utilizando **Apache Tomcat 7.0.47 (embebido vía Maven)**. Esta elección nos permite cumplir el requisito de "no afectar a la máquina madre", ya que Maven descarga el contenedor de Servlets localmente en la carpeta target/tomcat sin necesidad de instalar IIS ni modificar el registro de Windows. Además, usamos **JDBC** a través del driver de la base de datos H2.

Pregunta 2. Describe lo que es el modelo multicapa y los componentes que lo forman.

El **modelo multicapa (o arquitectura N-Tier)** es un patrón de diseño de software que organiza la aplicación en distintos niveles lógicos y físicos. Cada capa tiene una responsabilidad única y solo se comunica con las capas adyacentes. Esto garantiza la **Separación de Responsabilidades (SoC)**, facilitando el mantenimiento, la escalabilidad y la reutilización del código.

Los Componentes del Modelo Multicapa (Según CE1.1)

1. Cliente Ligero (Capa de Presentación - Frontend):
    - Es el navegador web (Chrome, Edge, Firefox). No contiene lógica de negocio pesada, solo interpreta HTML, CSS y JavaScript para mostrar la interfaz al usuario.

2. Servidor Web:
    - Software (como Apache HTTP Server o Nginx) que atiende las peticiones HTTP estáticas (imágenes, CSS, HTML puro) y las reenvía al servidor de aplicaciones si son dinámicas.
3. Servidor de Aplicaciones (Capa de Lógica de Negocio y Control):
    - Es el "cerebro" de la aplicación. Contiene los **Servlets** (Controladores) y la lógica de negocio. En J2EE, este componente se despliega en un contenedor como Tomcat.
4. Servidor de Base de Datos (Capa de Datos):
    - Sistema Gestor de Bases de Datos (SGBD) como MySQL, Oracle o, en nuestro caso, H2. Almacena la información de forma persistente y segura.

Las 3 Capas Lógicas (Patrón MVC)

En nuestro proyecto aplicamos el patrón **Modelo-Vista-Controlador (MVC)** sobre la arquitectura de 3 capas:

- **Capa de Presentación (Vista):** src/main/webapp/index.html y estilos.css.
- **Capa de Control (Controlador):** UsuarioServlet.java. Recibe la petición HTTP.
- **Capa de Negocio (Servicio):** UsuarioService.java. Aplica reglas de negocio.
- **Capa de Datos (Modelo y DAO):** UsuarioDAO.java (acceso JDBC) y Usuario.java (JavaBean).

Aplicación en nuestro Mini-Proyecto:

En nuestro proyecto-web-multicapa, la arquitectura física se distribuye así:

- **Cliente Ligero:** Tu navegador accediendo a http://localhost:8080/proyecto.
- **Servidor de Aplicaciones:** El plugin tomcat7-maven-plugin definido en nuestro pom.xml, que levanta un contenedor de Servlets en memoria.
- **Servidor de Base de Datos:** La base de datos **H2 en memoria** (jdbc:h2:mem:testdb), gestionada por el UsuarioDAO.java mediante sentencias SQL.

Pregunta 3. Define los siguientes conceptos: CSS, HTML, HTTP y TCP/IP.

Para que una aplicación web funcione, es necesario que intervengan protocolos de red y lenguajes de marcado. A continuación se definen y se explica su rol en nuestra aplicación.

1. TCP/IP (Transmission Control Protocol / Internet Protocol)

- **Definición:** Es el conjunto de protocolos de red fundamental de Internet. **IP** se encarga de direccionar y enrutar los paquetes de datos desde el origen hasta el

destino. **TCP** garantiza que la entrega sea fiable, ordenada y sin errores, estableciendo una conexión previa (handshake).

- **En nuestro proyecto:** Cuando escribes localhost:8080 en el navegador, TCP/IP establece un "túnel" fiable entre tu navegador (cliente) y el puerto 8080 de tu máquina local donde está escuchando Tomcat.

2. HTTP (HyperText Transfer Protocol)

- **Definición:** Es el protocolo de la capa de aplicación que define cómo se transmiten los mensajes entre el cliente y el servidor. Funciona bajo un modelo de **Petición-Respuesta** y es *sin estado* (stateless), lo que significa que cada petición es independiente.
- **En nuestro proyecto:** El navegador envía una petición GET /proyecto/usuarios HTTP/1.1. El Servlet (UsuarioServlet) procesa esta petición y devuelve una respuesta HTTP/1.1 200 OK con el cuerpo en HTML.

3. HTML (HyperText Markup Language)

- **Definición:** Es el lenguaje de marcado estándar utilizado para estructurar y dar significado semántico al contenido de una página web mediante etiquetas (<h1>, <p>, <a>). No es un lenguaje de programación, sino de descripción.
- **En nuestro proyecto:** Lo utilizamos en src/main/webapp/index.html para crear la estructura de la página de inicio, y también lo generamos dinámicamente dentro del PrintWriter de nuestro UsuarioServlet.java para mostrar la lista de usuarios.

4. CSS (Cascading Style Sheets)

- **Definición:** Es el lenguaje de diseño gráfico utilizado para definir y controlar la presentación visual (formato, colores, márgenes, tipografía) de un documento HTML. Permite separar el contenido (HTML) de la estética (CSS).
- **En nuestro proyecto:** Definimos los estilos en src/main/webapp/css/estilos.css (colores azules, bordes redondeados, sombras) y los enlazamos en el HTML mediante la etiqueta <link rel="stylesheet" href="css/estilos.css">.

Extra: CE1.7 - Funcionamiento de una sesión de aplicación en el modelo de programación web.

(Nota: Aunque no es una pregunta explícita, este objetivo es crucial en la rúbrica de la UF2405 y demuestra un conocimiento avanzado).

Dado que HTTP es un protocolo **sin estado (stateless)**, el servidor no sabe por defecto si dos peticiones provienen del mismo usuario. Para solucionar esto, en J2EE se utiliza el objeto **HttpSession**.

¿Cómo funciona en nuestro entorno J2EE?

1. **Primera petición:** Cuando un usuario accede por primera vez, el Servidor de Aplicaciones (Tomcat) crea un objeto HttpSession único en su memoria RAM y le asigna un identificador único llamado **JSESSIONID**.
2. **Envío al cliente:** Tomcat envía este JSESSIONID al navegador del cliente mediante una **Cookie** en la cabecera HTTP Set-Cookie.
3. **Peticiones posteriores:** En cada clic o recarga, el navegador devuelve automáticamente la Cookie con el JSESSIONID.
4. **Recuperación de estado:** El Servlet recibe el ID, busca la sesión en la memoria del servidor y recupera los datos del usuario (ej. su carrito de compra o su nombre de usuario logueado).

Ejemplo de código en nuestro Servlet:

```
// En el UsuarioServlet.java

HttpSession sesion = request.getSession(true); // Crea la sesión si no existe

sesion.setAttribute("ultimoAcceso", new java.util.Date());
```

Esto permite que la aplicación "recuerde" al usuario mientras su sesión esté activa (por defecto, 30 minutos de inactividad en Tomcat).